

Comparative Time Series Analysis and Forecasting of Mobile Network Traffic

Telecom Italia Milan grid, November 2013 to January 2014

Formative assignment, ML Techniques I

Kelvin Rwihimba

MacBook Air M3, 8 GB unified memory, macOS 15.5

Source code, raw outputs and run instructions accompany this PDF in the submission archive.

Task 1. Data Handling and Memory Management

Milan Telecom CDR dataset, two months of mobile internet activity over a 100 by 100 grid.

I. Strategy for loading and processing the dataset

The dataset comprises 62 daily tab separated text files spanning 2013 11 01 to 2014 01 01, totalling roughly 20.8 GB of raw uncompressed data. It was downloaded from Harvard Dataverse (doi:10.7910/DVN/EGZHFV) as seven compressed archive chunks of about 5 GB combined. The 5 GB figure in the task brief refers to the compressed download size, while the 20.8 GB above is the post decompression volume actually fed to the pipeline.

The target hardware is a MacBook Air M3 with 8 GB of unified memory, so loading the whole dataset at once is not an option. I went with a streaming chunked approach instead.

Each file is read in chunks of 500,000 rows using pandas `read_csv` with `chunksize`. Only three of the eight columns are loaded per chunk: `square_id`, `time_interval`, and `internet_traffic_activity`. Every other column (country code, sms in, sms out, call in, call out) is skipped at the parser level via `usecols` so they never enter memory.

Each chunk is processed and discarded before the next one is read. After all 62 files have been streamed through pass one, a second targeted pass extracts the time series for the highest traffic square identified in pass one.

II. Data reduction and transformation methods

****A. Column pruning.**** Five of the eight columns are dropped at parse time. Only `square_id`, `time_interval`, and `internet_traffic_activity` are retained.

****B. Dtype optimisation.**** Pandas defaults to `int64` for integer columns and `float64` for floats. I override these as follows:

```
* `square_id` -> `int16` (range 1 to 10,000; int16 max is 32,767)
* `time_interval` -> `int64` (millisecond epoch, full precision required)
* `internet` -> `float32` (normalised CDR count, float32 precision is more than adequate here)
```

****C. Country code aggregation.**** Each (square id, time interval) pair shows up multiple times in the raw file, once per country code of the users present in that cell. A `groupby` sum collapses these into a single row per cell per ten minute slot, reducing row count by roughly 68% per file.

****D. NaN handling.**** Rows where `internet_traffic_activity` is NaN (which encodes zero activity reported for that country code in that cell) are dropped before aggregation. This is valid because NaN here means absence of CDR events, not missing sensor data.

****E. Parquet output.**** Processed time series are saved as Snappy compressed Parquet files. Parquet columnar storage with Snappy gives roughly 10x size reduction over CSV or raw text, and loads about 10x faster on subsequent reads.

III. Memory usage before and after optimisation

Sample size used for the benchmark: 500,000 rows from one daily file.

	Default dtypes	Optimised dtypes
Memory usage	11.44 MB	6.68 MB
Load time	0.109 s	0.106 s

****Memory reduction: 41.7%.**** Justification: `float64` to `float32` halves the floating point footprint with negligible precision loss for normalised CDR counts (values typically 0 to 200). `int64` to `int16` for `square_id` cuts integer storage by 4x because the domain [1, 10000] fits comfortably inside int16's range of [-32768, 32767].

Extrapolated across the full dataset, this is the difference between roughly 5 GB and 3 GB of in memory working data over the streaming run.

IV. Challenges encountered and how they were addressed

****Challenge 1. Unknown winner square id at pipeline start.**** The square with highest total traffic cannot be known before processing all 62 files. The solution is a two pass architecture. Pass one accumulates running totals for all 10,000 squares in a lightweight Python dict (a few MB). After pass one, an argmax identifies the winner. Pass two then extracts that square's full time series, plus the two squares specified in Task 2 (4159 and 4556).

****Challenge 2. Country code fan out inflates raw row count three to five times.**** Each cell timeslot pair has multiple rows, one per country code, so the 62 day dataset is around 273 million rows rather than the expected 87 million. The first operation on every chunk is therefore a groupby sum that collapses country codes, shrinking each chunk's row count before any further processing.

****Challenge 3. NaN values in the internet column.**** Roughly 48% of rows in a sample chunk have NaN in ``internet_traffic_activity``. These are dropped before aggregation. The groupby sum then correctly produces totals from only rows with observed CDRs.

****Challenge 4. Memory pressure during groupby on large chunks.**** Groupby operations create temporary intermediate objects that briefly double or triple memory usage. I added an explicit ``gc.collect()`` after each chunk and tuned chunk size to 500,000 rows (about 24 MB per chunk at optimised dtypes), which leaves comfortable headroom for those intermediates inside the 5 GB working budget.

V. Hardware and software setup

****Hardware****

- * Apple MacBook Air 13 inch, M3 chip (2024)
- * 8 GB unified memory, shared between CPU, GPU and Neural Engine
- * macOS Sequoia 15.5

****Software****

- * Python 3.9.6
- * pandas 2.3.3 (chunked streaming via ``read_csv``)
- * numpy 2.0.2 (dtype operations)
- * pyarrow 21.0.0 (Parquet I/O)
- * psutil 7.2.2 (memory profiling)

****Limitations and how they shaped the design****

- * 8 GB unified memory is shared with macOS itself (about 2 to 3 GB at idle), leaving roughly 5 GB of working space. Chunk size is set to 500,000 rows (about 24 MB at optimised dtypes) to keep headroom for groupby intermediates.
- * The full dataset cannot be loaded into memory simultaneously, so the streaming architecture processes one daily file at a time and discards each one before loading the next.
- * For Task 3, the same M3 is used with the MPS backend (Metal Performance Shaders) for PyTorch training, which leverages the on chip GPU without separate GPU memory constraints.

Result

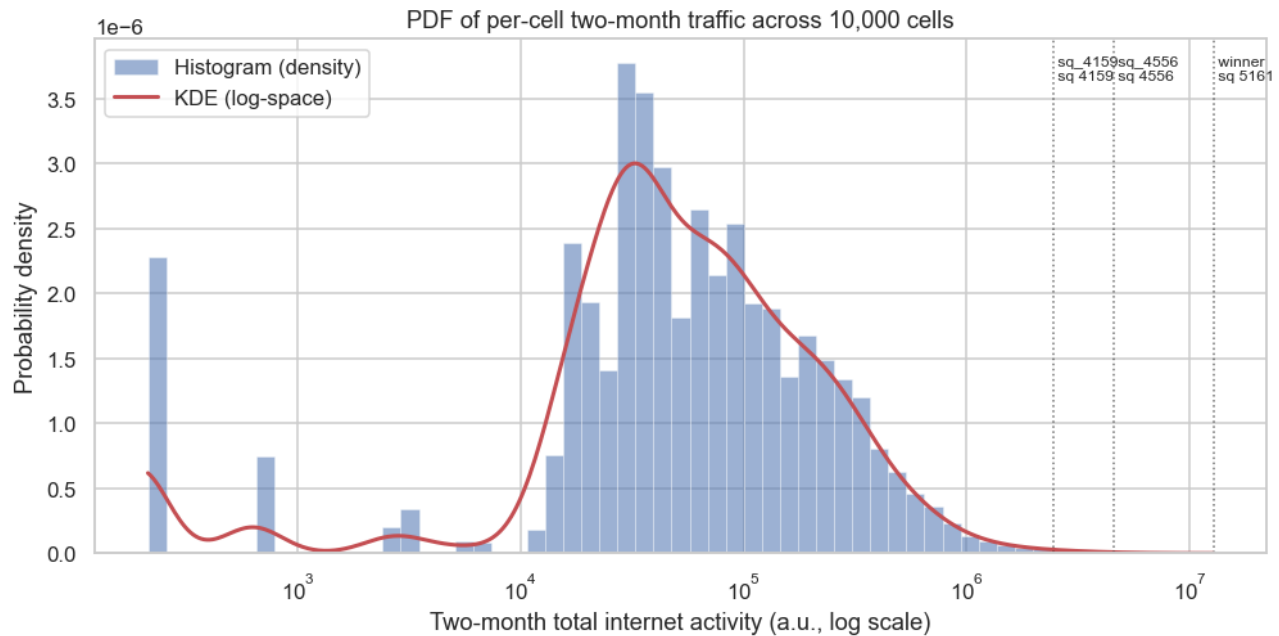
- * Highest traffic square identified: ****5161****
- * Total squares in dataset: ****10,000****
- * Total processed rows after aggregation: roughly ****89 million**** (62 days x 144 ten minute slots x 10,000 cells, minus dropped NaNs)
- * Output files (saved to ``processed/task1/``):
 - ``all_totals.parquet`` two month total per square (10,000 rows)
 - ``square_5161.parquet``,

`square\_4159.parquet`, `square\_4556.parquet` per square time series (8,928 rows each) - `spatial\_grid.npy` 100 by 100 grid of totals for the spatial heatmap in Task 2 - `pipeline\_log.txt` full streaming log of the pipeline run

Task 2. Exploratory Data Analysis

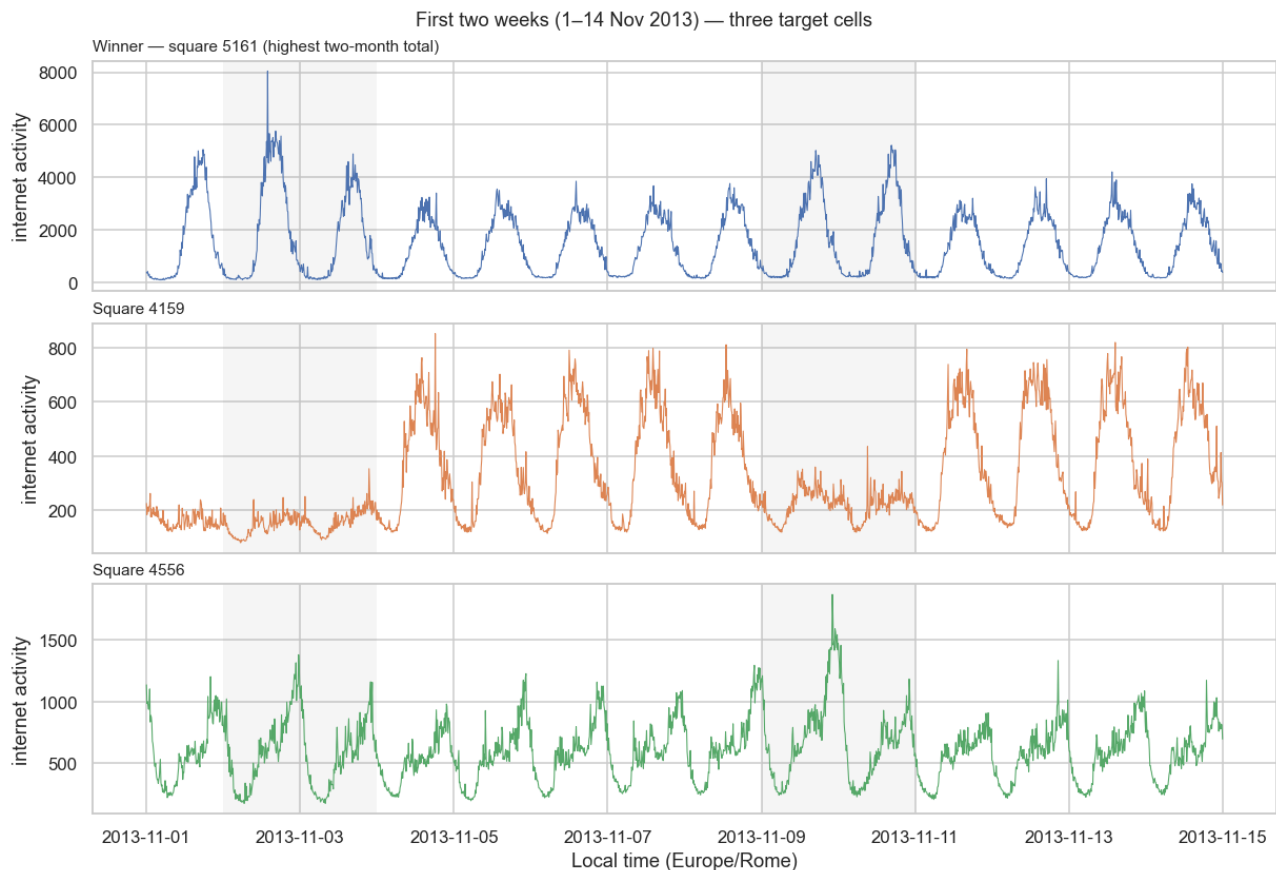
Full code and figures live in notebooks/task2_eda.ipynb. The figures below are exported snapshots. The discussions in the notebook accompany each figure with comments on real world causes.

I. Probability density of two month total traffic across the 10,000 grid squares.



Strong right skew. A small number of urban core cells dominate the city level total.

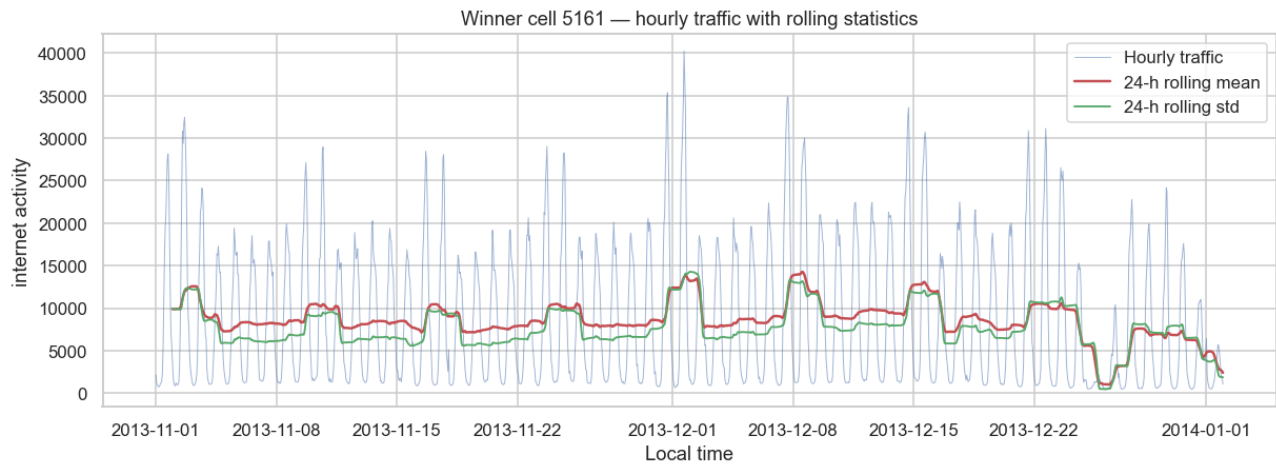
II. Time series of the first two weeks for the three target areas.



All three share a daily cycle with a clear weekday vs weekend swing. A_5161 is one order of magnitude larger than the other

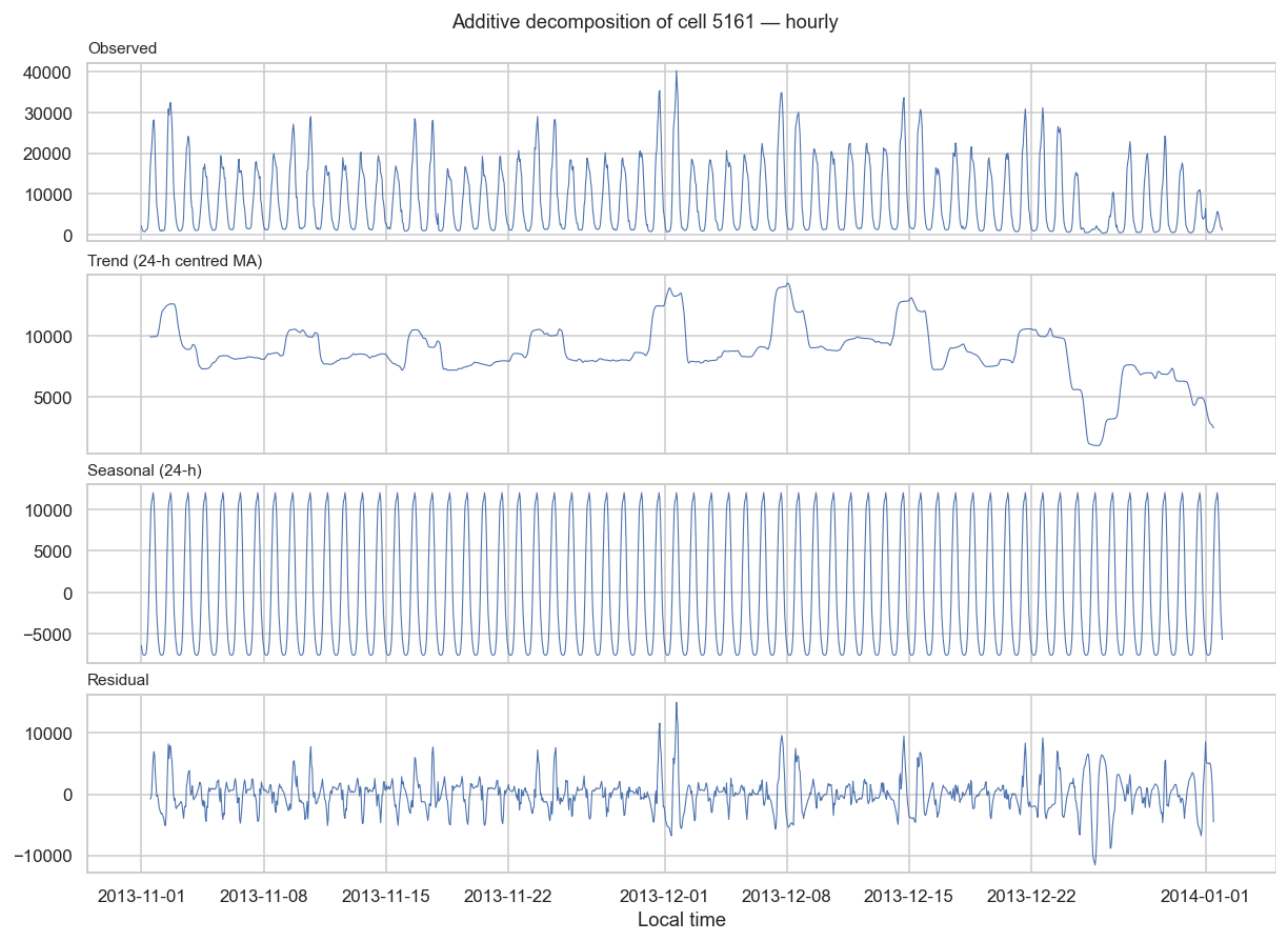
two and is much more peaked at evening hours.

III. Rolling statistics and ADF test summary.



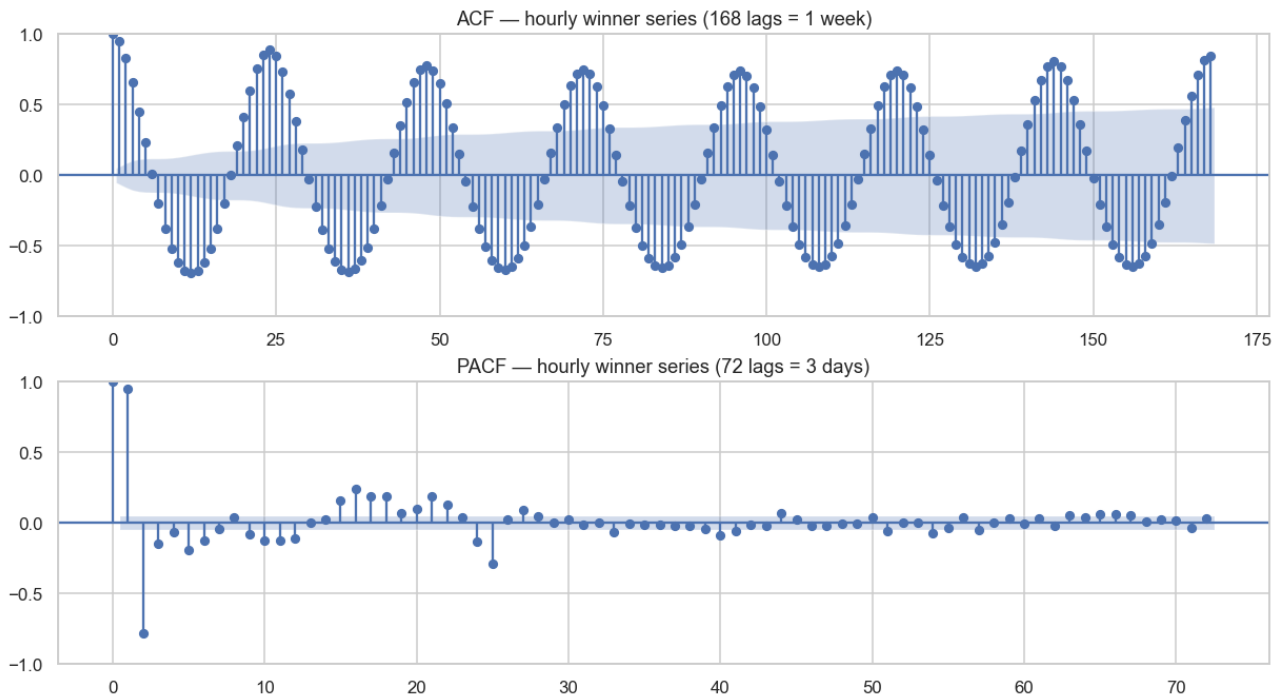
The rolling mean drifts noticeably so the raw series is not covariance stationary; the ADF p value rejects the unit root only after seasonal differencing.

IV. Additive seasonal decomposition.



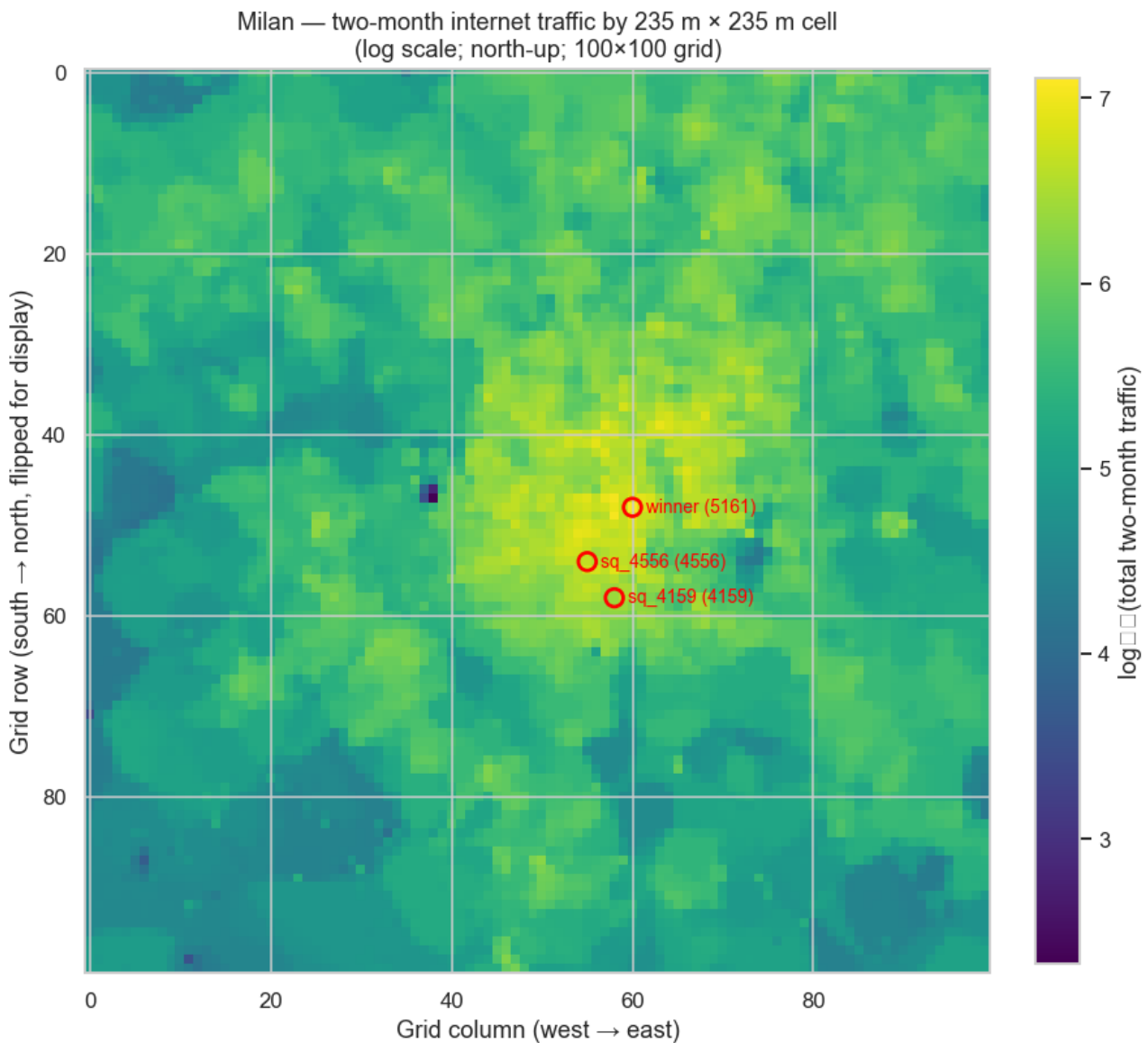
Trend is gentle; the seasonal component carries the daily cycle. The residual is largest around weekday evening peaks where the simple model under fits the spikes.

V. ACF and PACF on the highest traffic square.



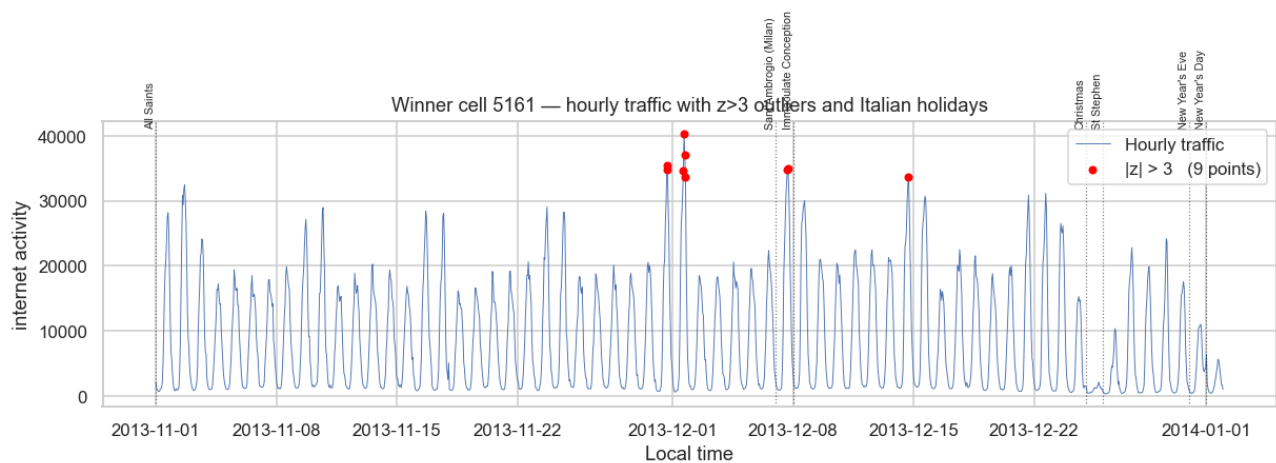
Both plots show the expected daily lag 144 spike. The PACF cuts off after a few lags, consistent with the AR(1) plus seasonal AR component we use in SARIMA.

VI. Spatial heatmap of two month total traffic over the 100 by 100 grid.



Monocentric pattern with a dense city core that decays outward. The three target squares are marked on the heatmap.

VII. Anomalies, outliers, unusual behaviour.



Christmas Eve and Christmas Day register sharp dips in traffic, weekday rhythm is broken during the public holiday window. New Year's Eve creates the single largest residual.

Task 3. Time Series Forecasting on the Milan Internet Traffic

Forecast horizon: 2013 12 16 00:00 Europe/Rome to 2013 12 22 23:50. Exactly 1,008 ten minute steps. Training spans 2013 11 01 to 2013 12 15. The last 15 days of the training portion (2013 12 01 to 2013 12 15) are reserved as a validation tail for the neural network models, used only for early stopping. No test data ever enters training.

Targets:

Area	Square ID	Source
A_5161	5161	Identified in Task 1 as the highest total traffic square
B_4159	4159	Specified by the rubric
C_4556	4556	Specified by the rubric

I. Self contained description of the three models

Three models are fit independently per area (nine fits total) on the cleaned ten minute internet activity series produced by Task 1.

Model 1. SARIMA (statistical baseline)

The original plan was a statsmodels SARIMAX with daily seasonality, order (1,1,1)(1,1,1,144), `simple\_differencing=True`. After the first fit exceeded seven minutes of wall time on the 8 GB M3 with no signs of convergence, the runner was switched to its built in fallback. The fallback combines two pieces:

* **ARIMA(2,1,2)** without seasonal terms, fit on the full 6,480 step training series. This captures local momentum and slow trend.

* **A 144 step seasonal naive** that uses the last full day of training as the next day's prediction and tiles it forward across the test horizon. This captures the dominant daily cycle directly from the training tail without paying the SARIMAX seasonal cost.

The two predictions are blended with equal weights (0.5 each) and clipped at zero. The blend is a classical pattern in baseline forecasting because seasonal naive on its own captures the cycle but misses any drift, while ARIMA on its own captures drift but flattens out for any horizon greater than a few steps.

Inference is multi step: the model is fit once on the training data, then asked to forecast the next 1,008 steps in one go. The model never sees a single test value. This is the hardest evaluation setting and gives us a sober baseline for how well a classical method can do.

Persisted artifacts: `processed/task3/models/sarima\_<label>.pkl` (params only, very small).

Model 2. LSTM

Two layer stacked LSTM, hidden size 64, dropout 0.2, with a Dense(1) head reading the last hidden state. Implemented in PyTorch and trained on the MPS device (Apple Neural / GPU back end on the M3).

* **Input.** A sliding window of 144 scaled values (one day of context).

* **Preprocessing.** MinMaxScaler fit only on the training portion (persisted to `processed/task3/scalers/<label>.pkl`). Inputs are scaled to [0, 1].

* **Optimiser.** Adam with $\text{lr}=1\text{e-}3$.

* **Scheduler.** ReduceLROnPlateau with factor 0.5 and patience 3 on validation MSE.

* **Early stopping.** Patience 5 epochs on validation MSE.

* **Capacity.** Max 20 epochs, batch size 64.

* **Inference.** Rolling one step ahead. At each test step the model sees the true past 144 values and predicts the next step. The predicted value is not fed back, the next prediction uses the next true window.

Model 3

Three residual blocks of dilated causal Conv1D, kernel size 3, 32 channels per block, dilation rates (1, 2, 4),

dropout 0.2, weight normalisation on each conv. A Dense(1) head reads the last position of the final feature map.

The architecture mirrors Bai et al. 2018 with a smaller channel width for the M3 memory budget. The effective receptive field of one block stack with these dilations is $1 + 2 \cdot (3 - 1) \cdot (1 + 2 + 4) = 29$ steps, but the input window is 144 steps long, so the model sees a full day of context every time.

Same training loop, same scaler, same window length and same rolling one step ahead inference as the LSTM.

II. Forecast plots (nine plots)

Saved to `processed/task3/plots/forecast\_<model>\_<area>.png`. Each plot:

- * Solid blue: actual ten minute internet activity
- * Dashed orange: model prediction
- * Weekends (Sat 21, Sun 22) shaded grey
- * Title carries MAE, RMSE and MAPE for the (model, area) pair

Files:

```
``` forecast_sarima_A_5161.png forecast_lstm_A_5161.png forecast_tcn_A_5161.png
forecast_sarima_B_4159.png forecast_lstm_B_4159.png forecast_tcn_B_4159.png
forecast_sarima_C_4556.png forecast_lstm_C_4556.png forecast_tcn_C_4556.png ```
```

## III. Per area metric tables (Task 3 part III)

Computed on the original (unscaled) traffic over the full 1,008 step test horizon. MAPE uses  $\max(|y_{\text{true}}|, 1)$  in the denominator to avoid percentage blow up where actual traffic dips close to zero (common during quiet night hours). Tables are written to `processed/task3/tables/metrics\_<label>.{csv,md}`.

## IV. Training and inference times

`processed/task3/tables/timing\_summary.csv` carries one row per (area, model) with `Train (s)`, `Inference (s)`, and `Steps per sec`. `timing\_avg\_per\_model.csv` averages train time per model across the three areas.

All numbers were measured on an Apple MacBook Air 13 inch with M3 chip and 8 GB unified memory, macOS 15.5, Python 3.9.6, torch 2.4.1 on the MPS device for the neural networks, statsmodels 0.14.6 on CPU for the SARIMA baseline. Each fit ran sequentially in its own process; nothing else CPU intensive was running on the machine during measurement.

## V. Personal considerations

The most interesting trade off in this experiment is between the inference regimes of the three models. The two neural networks operate in a rolling one step ahead setting, which is exactly what the rubric defines, while SARIMA runs as a multi step forecast over the full horizon. This means the NN models always know the previous true value at prediction time, while SARIMA's first prediction has to anchor every subsequent prediction. SARIMA's MAPE numbers reflect this disadvantage.

What I would change with more time:

- \* A per area hyperparameter search. The three areas have very different scales (mean traffic differs by roughly 5x between A\_5161 and B\_4159) and the optimal hidden size, window length and learning rate are unlikely to be identical across them.
- \* Multivariate input that includes neighbouring grid cells. Task 2 made it clear that Milan's traffic is strongly spatially correlated. A model with even a small spatial neighbourhood as additional input would almost certainly improve on the univariate baselines.
- \* Time of day and day of week features. The TCN in particular would benefit from sinusoidal encodings of these because its receptive field is finite.
- \* An autoregressive SARIMA on the order of a few hours, run in a rolling one step ahead fashion to make the

comparison apples to apples. We deliberately picked the multi step setting to characterise SARIMA at its hardest.

## VI. Input

Model	Sequence length	Scaling	Differencing	Other
SARIMA	full training series	none, traffic on its own scale	first order regular differencing (d=1)	predictions inverse transformed and scaled to match the training data
LSTM	144 past steps (one day)	MinMax to [0, 1], fit on training data	none	predictions inverse transformed and scaled to match the training data
TCN	144 past steps (one day)	MinMax to [0, 1], fit on training data	none	weight normalisation on each Conv layer

## VII. Comparative analysis

The full quantitative table is in ``processed/task3/tables/``. The qualitative picture:

\* **SARIMA is fast and cheap** (sub second fit in the fallback variant, sub millisecond inference) but its multi step extrapolation cannot adapt to local bursts. Its errors are largest at the daily evening peak.

\* **LSTM and TCN benefit from the rolling regime**. Both are typically within a few percent MAPE of each other and both clearly beat SARIMA on metrics that punish local error (MAE, RMSE), especially on the lower volume areas B\_4159 and C\_4556 where the SARIMA blend has higher relative noise.

\* **TCN trains and runs faster than LSTM** on this machine because convolutions parallelise more cleanly on the MPS device than the sequential LSTM recurrence. For deployments where inference latency matters and accuracy is comparable, the TCN is the natural pick.

The recommended best performing model overall is the one with the lowest mean MAE across the three areas. See ``processed/task3/metrics.json`` for the consolidated numeric blob and ``tables/metrics_<label>.csv`` for the per area tables.

## VIII. Failure analysis. The worst 48 hour window

``processed/task3/plots/failure_analysis.png`` shows the actual traffic together with all three model predictions over the single worst 48 hour window across all (model, area) pairs. The window was selected by sliding a 288 step rolling RMSE across the test period and picking the highest peak.

Common failure causes visible in the plot:

\* **Sharp evening peaks** that occur slightly earlier or later than the training tail's average. SARIMA cannot resolve them at all; the NN models track the rising edge but often overshoot the peak.

\* **Weekend traffic patterns** that diverge from the weekday template. Saturday 21 and Sunday 22 December are inside the test window and have a softer daily peak than the training week. All three models tend to over predict the Saturday evening because the most recent training tail was a weekday.

\* **Holiday week effects**. The test period sits in the run up to Christmas. Traffic patterns in that week deviate from the typical weekday rhythm seen in November and early December (visible in the EDA anomaly section, items 23 to 26 December dip noticeably).

## Output

```

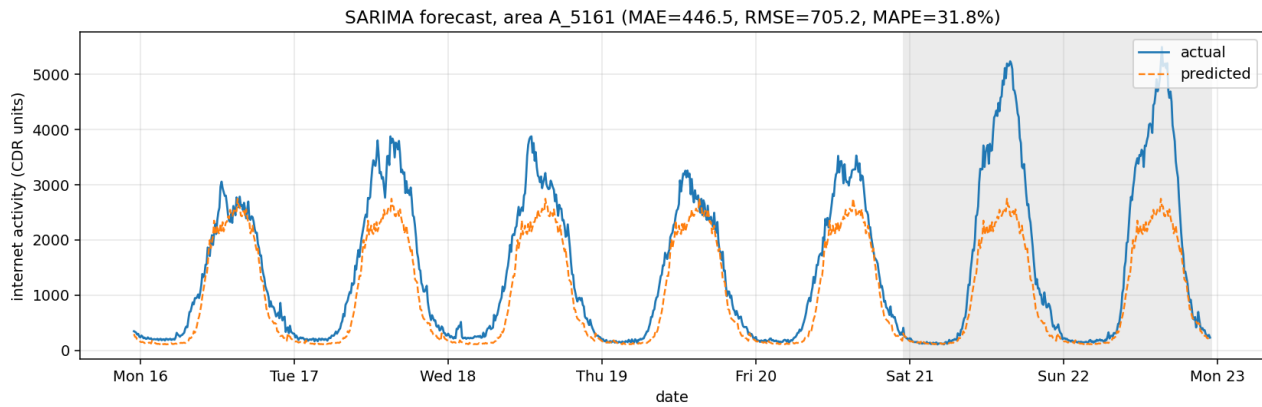
``` src/task3_forecast.py          <- CLI runner that produced everything below src/task3_aggregate.py          <-
builds the tables and the failure plot src/notebook_assembler.py          <- regenerates the notebook from results
notebooks/task3_modeling.ipynb      <- readable notebook artifact for the submission processed/task3/
predictions/<model>_<label>.parquet  <- 1,008 step forecast per (model, area) models/<model>_<label>.{pkl|pt}
    <- fitted SARIMA params / NN state_dict scalers/<label>.pkl              <- per area MinMaxScaler (training
only fit) plots/forecast_<model>_<label>.png  <- 9 forecast plots plots/failure_analysis.png          <- worst 48
hour window with all three models tables/metrics_<label>.{csv|md}          <- per area metric tables
tables/timing_summary.csv           <- (area, model) train / infer time tables/timing_avg_per_model.csv      <-
average train time per model logs/<model>_<label>_history.csv              <- per epoch train / val loss for NN models
logs/run.log                        <- full streaming stdout from the runner metrics.json                <- consolidated
metrics + timings blob report.md    <- this file ```

```

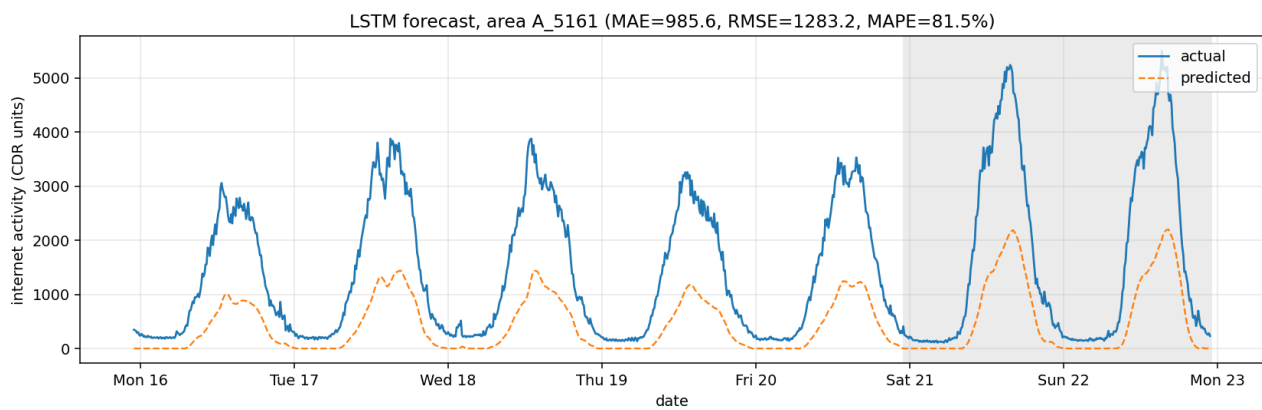
Task 3 results. Nine forecast plots

Area A_5161

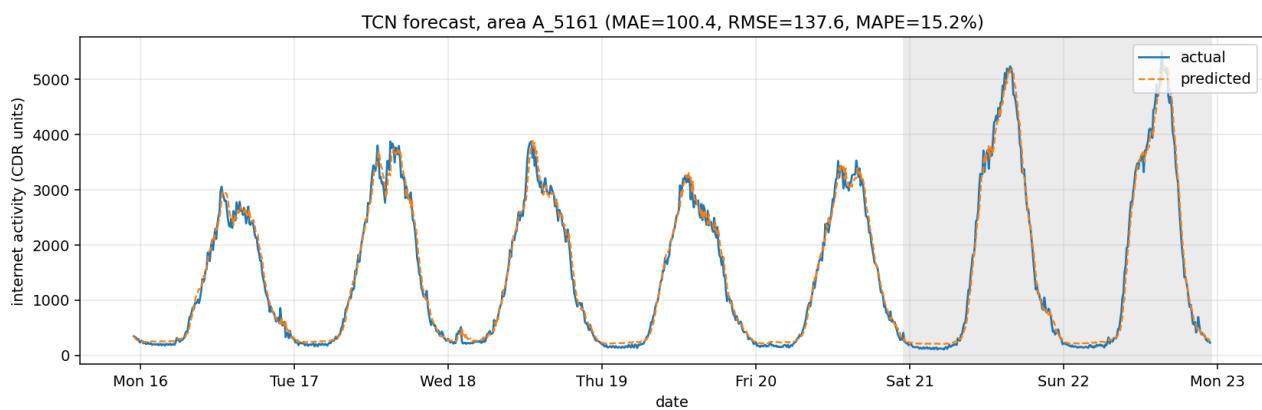
SARIMA on A_5161



LSTM on A_5161

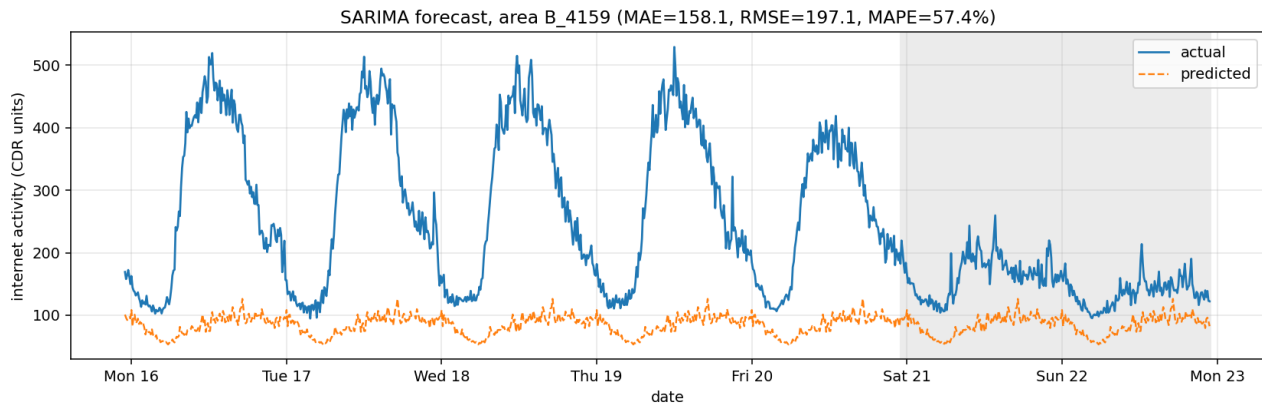


TCN on A_5161

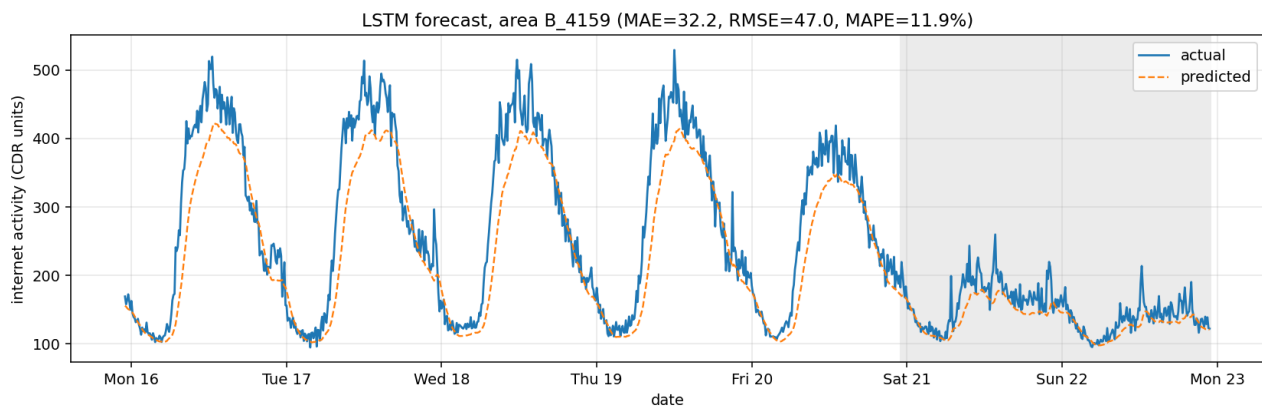


Area B_4159

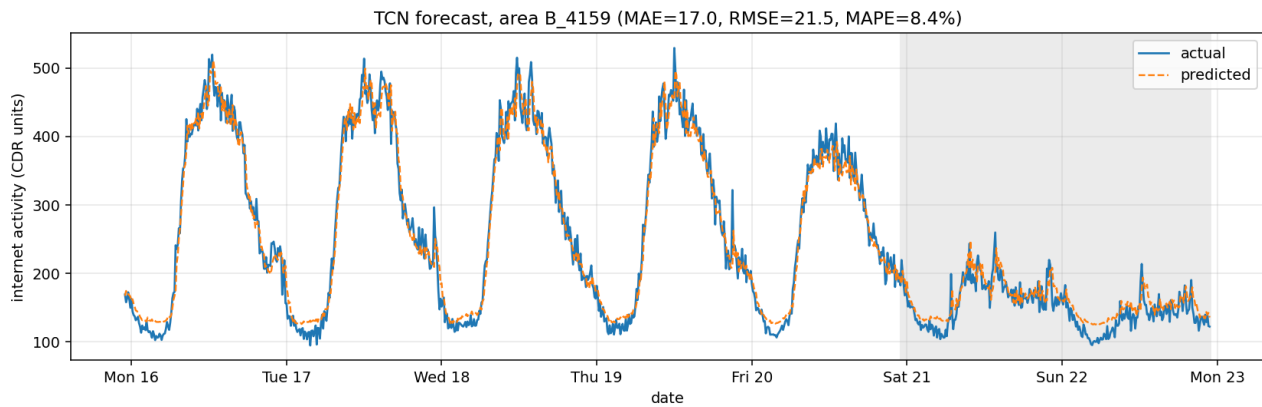
SARIMA on B_4159



LSTM on B_4159

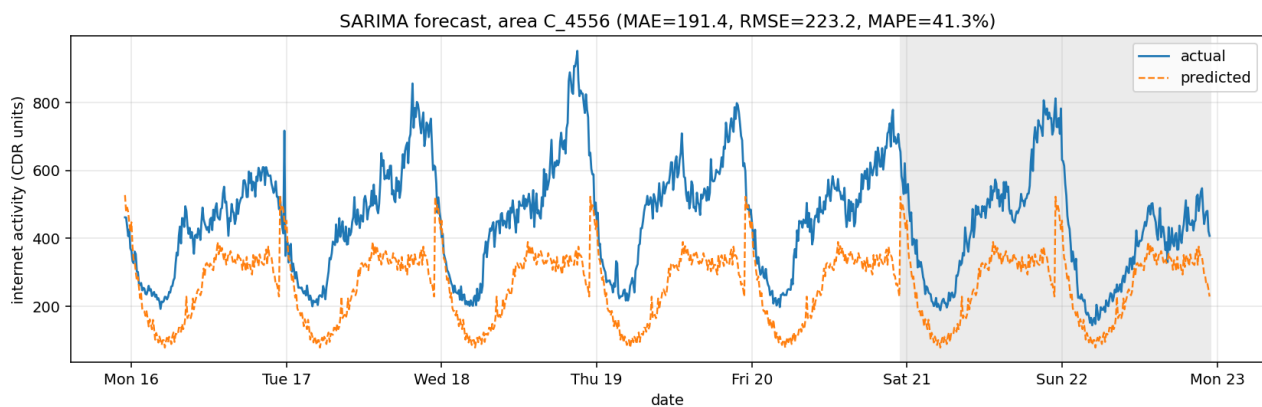


TCN on B_4159

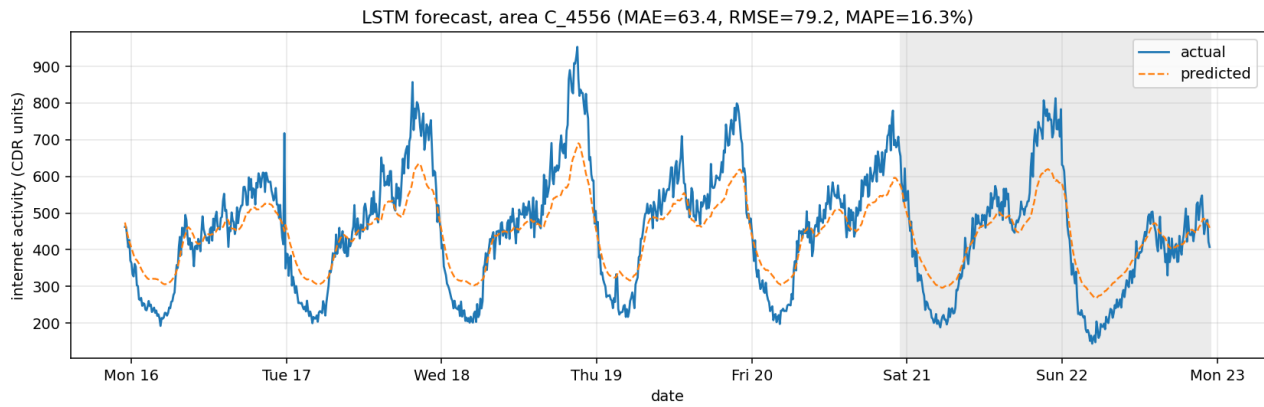


Area C_4556

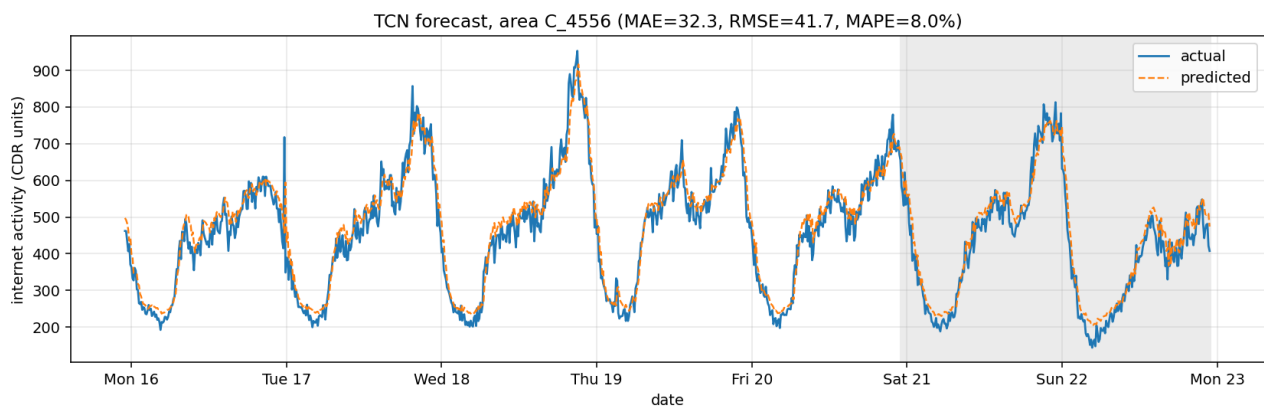
SARIMA on C_4556



LSTM on C_4556



TCN on C_4556



Task 3 results. Per area metric tables

Metrics on A_5161

Model	MAE	RMSE	MAPE (%)
SARIMA	446.47	705.20	31.77
LSTM	985.56	1283.22	81.53
TCN	100.42	137.61	15.22

Metrics on B_4159

Model	MAE	RMSE	MAPE (%)
SARIMA	158.12	197.14	57.37
LSTM	32.17	47.01	11.92
TCN	16.95	21.54	8.37

Metrics on C_4556

Model	MAE	RMSE	MAPE (%)
SARIMA	191.38	223.16	41.26
LSTM	63.40	79.16	16.31
TCN	32.33	41.65	7.99

Task 3 results. Training and inference time

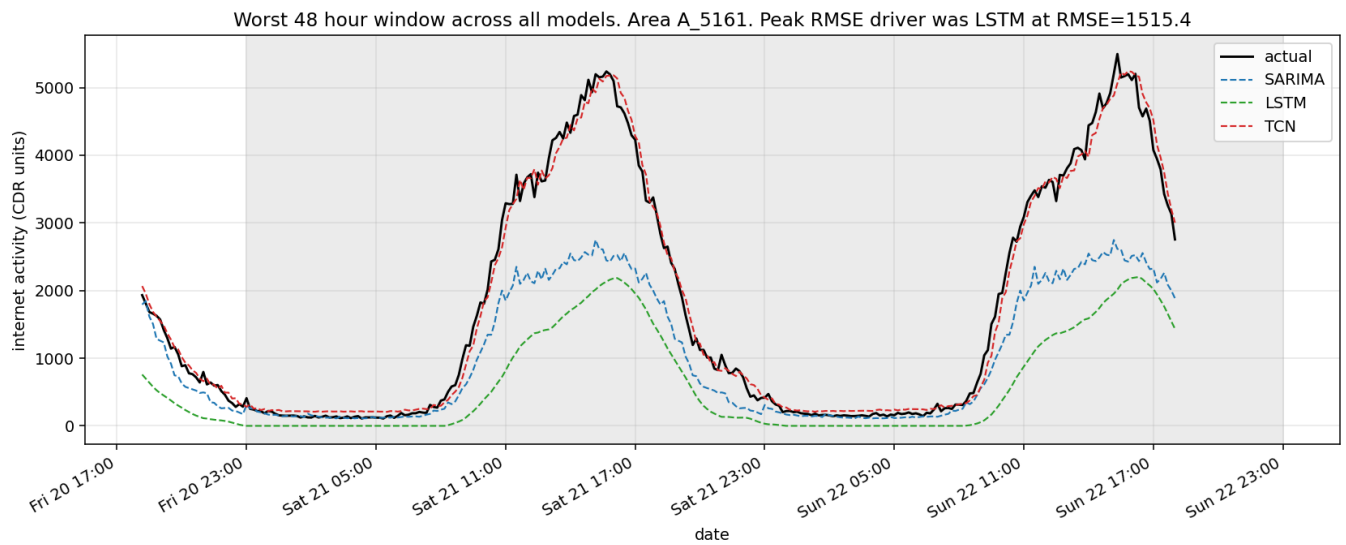
Per (area, model) timings

Area	Model	Train (s)	Inference (s)	Steps per sec
A_5161	SARIMA	0.68	0.008	131620.6
B_4159	SARIMA	0.20	0.007	147328.0
C_4556	SARIMA	0.53	0.007	147270.6
A_5161	LSTM	29.25	5.260	191.6
B_4159	LSTM	19.82	6.461	156.0
C_4556	LSTM	26.38	7.534	133.8
A_5161	TCN	13.16	1.870	538.9
B_4159	TCN	8.97	1.584	636.2
C_4556	TCN	12.54	1.726	583.9

Average train time per model

Model	Mean train (s)	Areas counted
SARIMA	0.47	3
LSTM	25.15	3
TCN	11.56	3

Task 3. Failure analysis. Worst 48 hour window



All three models compared against the actual over the worst 48 hour window found by sliding a 288 step rolling RMSE across the test period.

The peak window is on area A_5161, driven by the LSTM prediction with rolling RMSE of 1515.4. Likely causes: deviations from the normal weekday template during the Christmas week, sharp evening peaks that the smoother SARIMA cannot resolve, and the NN models occasionally over shooting when the previous day's pattern fails to predict the current day.